Experiment 6: Mutual Exclusion Algorithms

Date Of Performance:
Date of Submission:

Aim
To study, implement, and analyze mutual exclusion algorithms that ensure safe and coordinated access to shared resources in concurrent and distributed computing environments.

Objectives

1. To understand the critical section problem and the need for mutual exclusion.
2. To study classical and distributed mutual exclusion algorithms.
3. To implement selected mutual exclusion algorithms and observe their behavior.
4. To analyze algorithm performance in terms of fairness, deadlock freedom, and starvation prevention.
5. To compare algorithms based on message complexity and synchronization efficiency.

Scope of the Experiment

Scope 1: Conceptual Understanding of Mutual Exclusion

This scope focuses on understanding the critical section problem and the necessity of mutual exclusion in concurrent systems. It examines issues such as race conditions, deadlock, and starvation. The scope emphasizes safety and liveness properties required for correct synchronization. Theoretical foundations form the basis for algorithm analysis.

Scope 2: Algorithm Implementation and Behavior Analysis

This scope covers the implementation of selected mutual exclusion algorithms in a simulated environment. It covers the logical working of algorithms such as Peterson's Algorithm, Lamport's Bakery Algorithm, and distributed mutual exclusion protocols.

Scope 3: Performance and Comparative Evaluation

This scope involves comparing mutual exclusion algorithms based on performance metrics such as message complexity, waiting time, and scalability. It analyzes efficiency in both centralized and distributed settings. The scope highlights trade-offs between simplicity and communication overhead. Results support informed selection of algorithms for real-world systems.

Literature Survey

The concept of mutual exclusion was first formalized by **Edsger W. Dijkstra (1965)** to address synchronization issues in multiprogramming systems. The concept of mutual exclusion was first formalized by Edsger W. Dijkstra in 1965 to address synchronization issues in multiprogramming systems. He introduced the critical section problem, where multiple processes attempt to access shared resources simultaneously, leading to race conditions and inconsistent system states. Early solutions such as Dekker's Algorithm focused on software-based synchronization techniques without relying on hardware support. These approaches aimed to ensure that only one process could execute in the critical section at any given time, forming the foundation of process synchronization in operating systems.

Subsequently, Leslie Lamport in 1974 made significant contributions by introducing logical clocks, which provided a method to order events in distributed systems where no global clock exists. Building upon this, he proposed the Bakery Algorithm, which ensures fairness by assigning numbers to processes similar to customers waiting in a queue. This approach guarantees first-come-first-served access to the critical section and eliminates starvation. Later, Gary L. Peterson introduced Peterson's Algorithm in 1981, a simple yet effective solution for achieving mutual exclusion between two

processes using shared variables and turn-taking mechanisms.

Conceptual Understanding of Mutual Exclusion

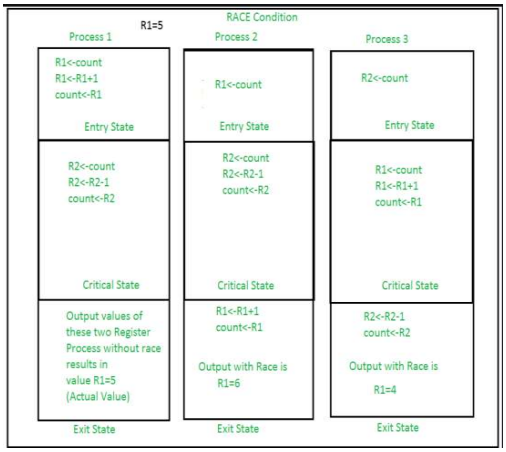
The critical section problem arises when multiple processes concurrently access shared resources, leading to race conditions and inconsistent system states. Early solutions emphasized software-based synchronization, avoiding hardware dependence.

Researchers such as Lamport (1974) introduced logical clocks to reason about event ordering, establishing theoretical foundations for distributed synchronization. Later studies focused on safety properties (ensuring only one process enters the critical section at a time) and liveness properties (ensuring progress and freedom from starvation). These theoretical principles remain central to modern operating systems and distributed computing.

As systems evolved, hardware-based synchronization techniques were introduced to improve efficiency. Atomic instructions such as Test-and-Set and Compare-and-Swap allowed processes to lock critical sections with minimal overhead. While these methods improved performance, they introduced challenges such as busy waiting, where processes continuously check for lock availability, potentially wasting CPU resources. To address these issues, higher-level synchronization constructs such as semaphores and monitors were developed, enabling better control over process coordination without excessive CPU usage.

In distributed systems, mutual exclusion becomes more complex due to the absence of shared memory and the presence of communication delays. Algorithms such as Lamport's Distributed Mutual Exclusion Algorithm, Ricart–Agrawala Algorithm, and Token Ring Algorithm were developed to handle synchronization through message passing. These approaches focus on minimizing message overhead while ensuring fairness, safety, and liveness. Modern research continues to optimize these algorithms for scalability, fault tolerance, and performance in large-scale distributed environments such as cloud computing and multi-hospital data-sharing systems.

From a theoretical perspective, mutual exclusion is governed by key properties including safety, liveness, and fairness. Safety ensures that no two processes are in the critical section simultaneously, while liveness guarantees that every process requesting access will eventually be granted entry. Fairness prevents indefinite postponement or starvation of any process. These principles remain central to the design of modern operating systems, concurrent programming models, and distributed computing architectures, ensuring reliable and consistent access to shared resources.



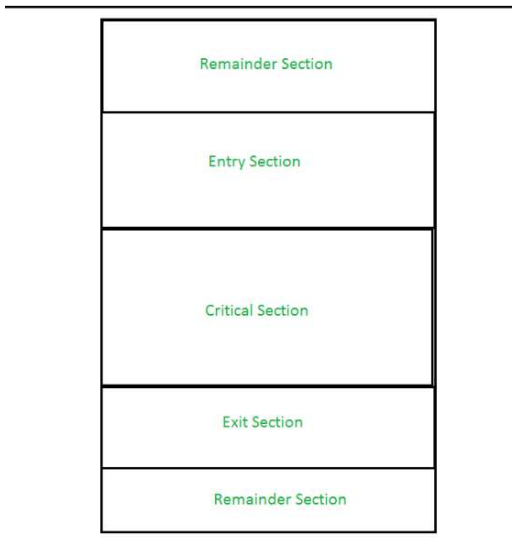


Fig .Critical Section Diagram

Algorithm Implementation and Behavior Analysis

Algorithm Studied: Critical Section Problem Model Algorithm

Name :Critical Section Synchronization Model Input Variables

- n: Number of concurrent processes
- CS: Shared critical section
- Pi: Process identifier (i = 1 to n)

Output Variables

- Mutual exclusion guaranteed access to CS
- Correct execution order without race conditions

Algorithm (Conceptual Steps)

1. Each process Pi repeatedly executes **entry section**, **critical section**, **exit section**, and **remainder section**.
2. Entry section checks whether the critical section is free.
3. If free, process enters the critical section.
4. After execution, process exits and allows other waiting processes to proceed.

Algorithm 1: Peterson's Algorithm

Algorithm Name :Peterson's Mutual Exclusion Algorithm Input Variables

- flag[2]: Boolean array indicating intention to enter critical section
- turn: Integer indicating which process has priority

Output Variables

- Exclusive access to critical section
- Deadlock-free execution for two processes

Algorithm Steps

1. Process Pi sets flag[i] = true.
2. Process sets turn = j (other process).
3. While flag[j] == true and turn == j, wait.
4. Enter critical section.
5. On exit, set flag[i] = false

Algorithm 2: Lamport's Bakery Algorithm
Input Variables

- choosing[n]: Boolean array
- number[n]: Integer array for ticket numbers

Output Variables

- Mutual exclusion for n processes
- Fair access to critical section

Algorithm Steps

1. Process sets choosing[i] = true.
2. Assigns number[i] = max(number) + 1.
3. Sets choosing[i] = false.
4. Waits until all other processes either finish choosing or have a higher ticket.
5. Enters critical section.
6. Sets number[i] = 0 on exit.

Algorithm 3: Lamport's Distributed Mutual Exclusion Algorithm

Input Variables

- Ti: Logical timestamp
- REQ: Request message
- ACK: Acknowledgment message

Output Variables

- Exclusive access to distributed critical section

Algorithm Steps

1. Process sends REQ(Ti, Pi) to all other processes.
2. Each receiver replies with ACK.
3. Requests are queued based on timestamp ordering.
4. Process enters critical section when all ACKs are received and its request is at queue head.
5. On exit, sends release message.

Algorithm 4: Ricart–Agrawala Distributed Mutual Exclusion Algorithm

Input Variables

- REQ(Ti, Pi)
- REPLY
- Logical clock Ti

Output Variables

- Reduced message overhead
- Fair critical section access

Algorithm Steps

1. Process broadcasts REQ message with timestamp.
2. Receiver replies immediately if not requesting CS or has higher timestamp.
3. Otherwise, reply is deferred.
4. Process enters CS after receiving all replies.
5. Deferred replies are sent on exit.

Algorithm 5: Suzuki–Kasami Suzuki–Kasami Token-Based Distributed Mutual Exclusion Algorithm

Input Variables

- Token
- RN[i]: Request number
- LN[i]: Last serviced request number

Output Variables

- Token-controlled critical section access
- Reduced message complexity

Algorithm Steps

1. Process broadcasts request if it does not have the token.
2. Token holder checks pending requests.
3. Token is passed to next eligible requester.
4. Only token holder enters critical section.
5. Token circulation ensures fairness.

Programming Language Used

2.2.1.1 Libraries Used

1. Multithreading Library (<thread>)

Purpose:
Used to create and manage multiple concurrent processes (threads) executing simultaneously.

Syntax:

```
#include <thread>
std::thread t(function_name, arguments);
```

Usage in Program:
Used to simulate multiple processes competing for the critical section in Peterson's and Bakery algorithms.

2. Atomic Library (<atomic>)

Purpose:
Provides atomic operations to avoid race conditions during shared variable access.

Syntax:

```
#include <atomic>
std::atomic<int> var;
std::atomic<bool> flag;
```

Usage in Program:
Used for shared variables like flag, turn, choosing, and number to ensure safe concurrent execution.

3. Queue Library (<queue>)

Purpose:
Used to maintain ordered requests in distributed mutual exclusion.

Syntax:

```
#include <queue>
std::queue<Type> q;
```

Usage in Program:
Used in Lamport's Distributed Algorithm to store and process requests in FIFO order.

4. Input-Output Library (<iostream>)

Purpose:
Used for displaying output and taking input from the user.

Syntax:

```
#include <iostream>
std::cout << "Message";
std::cin >> variable;
```

Usage in Program:
Displays process execution steps and critical section access messages.

- **2.2.1.2 Classes / Methods / Constructs Used**

1. std::thread

Purpose:
Creates concurrent execution units (threads) representing processes.

Syntax:

```
std::thread t(function, args);
```

Usage in Program:
Used to run multiple processes simultaneously in all mutual exclusion algorithms

2. flag[i] = true

Purpose:
Indicates that a process wants to enter the critical section.

Syntax:

```
flag[i] = true;
```

Usage in Program:
Used in Peterson's Algorithm to signal entry request.

3. turn

Purpose:
Controls which process gets priority when both request access.

Syntax:

```
turn = j;
```

Usage in Program:
Ensures fairness in Peterson's Algorithm.

4. Busy Waiting Loop

Purpose:
Forces a process to wait until it is safe to enter the critical section.

Syntax:

```
while(flag[j] && turn == j);
```

Usage in Program:
Prevents simultaneous entry into the critical section.

5. choosing[i]

Purpose:
Indicates that a process is selecting its ticket number.

Syntax:

```
choosing[i] = true;
```

Usage in Program:
Used in Bakery Algorithm to avoid inconsistent reads.

6. number[i] = max(number[]) + 1

Purpose:
Assigns a unique ticket number for ordering processes.

Syntax:

```
number[i] = max(number) + 1;
```

Usage in Program:
Ensures fairness and first-come-first-served execution.

7. Ticket Comparison Condition

Purpose:
Ensures the process with the smallest ticket enters first.

Syntax:

```
while(number[j] != 0 &&
    (number[j] < number[i] ||
    (number[j] == number[i] && j < i)));
```

Usage in Program:
Controls entry into the critical section in Bakery Algorithm.

8. std::queue<Request>

Purpose:
Maintains ordered requests in distributed systems.

Syntax:

```
std::queue<Request> q;
```

Usage in Program:
Used in Lamport's Distributed Algorithm for request management.

9. push()

Purpose:
Adds a request to the queue.

Syntax:

```
q.push(request);
```

Usage in Program:
Simulates sending a request for critical section access.

10. pop()

Purpose:
Removes a completed request from the queue.

Syntax:

```
q.pop();
```

3. IMPLEMENTATION

Algorithm Used in C++

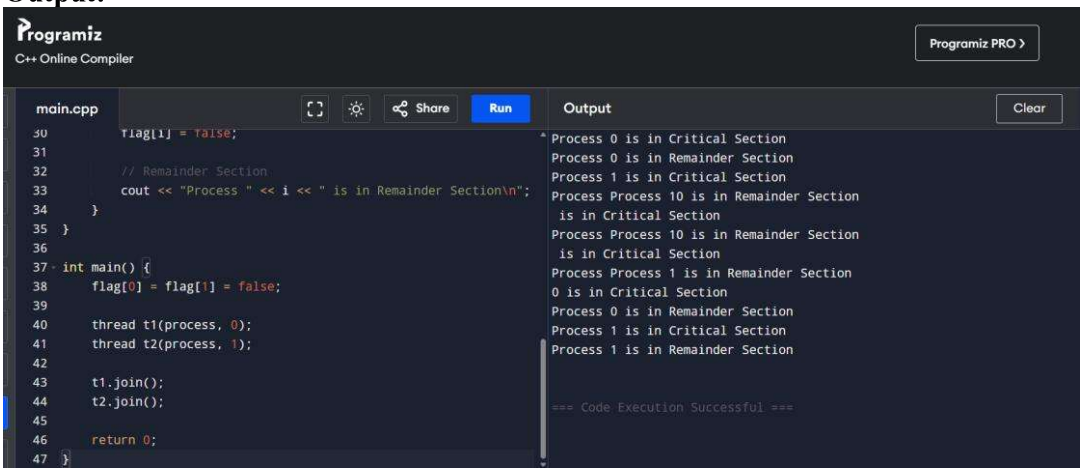
Peterson's Mutual Exclusion Algorithm (Two Processes)

```
#include <iostream>
#include <thread>
#include <atomic>
using namespace std;
atomic<bool> flag[2];   // Indicates interest of processes
atomic<int> turn;       // Whose turn is it
// Critical Section Function
void critical_section(int i) {
    cout << "Process " << i << " is in Critical Section\n";
}
// Process Function
void process(int i) {
    int j = 1 - i; // Other process
    for(int k = 0; k < 3; k++) { // Repeat multiple times
        // Entry Section
        flag[i] = true;
        turn = j;
        while(flag[j] && turn == j); // Busy waiting
        // Critical Section
        critical_section(i);
        // Exit Section
        flag[i] = false;
        // Remainder Section
        cout << "Process " << i << " is in Remainder Section\n";
    }
}
int main() {
    flag[0] = flag[1] = false;
    thread t1(process, 0);
    thread t2(process, 1);
    t1.join();
    t2.join();
    return 0;
}
```

Command: Run

Platform: Online Compiler

Output:



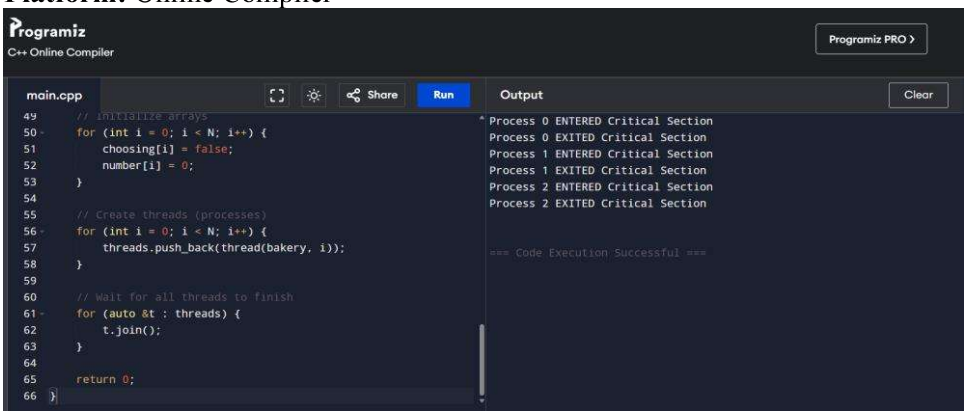
2. Lamport's Bakery Algorithm (C++)

Lamport's Bakery Mutual Exclusion Algorithm (Multiple Processes)

C++ Code

```
#include <iostream>
#include <thread>
#include <vector>
#include <atomic>
#include <chrono>
#include <algorithm>
using namespace std;
const int N = 3;  // Number of processes
atomic<bool> choosing[N];
atomic<int> number[N];
// Bakery Algorithm Function
void bakery(int id) {
    // Step 1: Choosing a number
    choosing[id] = true;
    int maxnum = 0;
    for (int i = 0; i < N; i++) {
        maxnum = max(maxnum, number[i].load());
    }
    number[id] = maxnum + 1;
    choosing[id] = false;
    // Step 2: Wait until it's this process's turn
    for (int j = 0; j < N; j++) {
        while (choosing[j]); // Wait if process j is choosing
        while (number[j] != 0 &&
              (number[j] < number[id] ||
              (number[j] == number[id] && j < id)));
    }
    // 🔹 Critical Section
    cout << "Process " << id << " ENTERED Critical Section" << endl;
    this_thread::sleep_for(chrono::milliseconds(500));
    cout << "Process " << id << " EXITED Critical Section" << endl;
    // Step 3: Exit Section
    number[id] = 0;
}
int main() {
    vector<thread> threads;
    // Initialize arrays
    for (int i = 0; i < N; i++) {
        choosing[i] = false;
        number[i] = 0;
    }
    // Create threads (processes)
    for (int i = 0; i < N; i++) {
        threads.push_back(thread(bakery, i));
    }
    // Wait for all threads to finish
    for (auto &t : threads) {
        t.join();
    }
    return 0;}
```

Output
Command: Run
Platform: Online Compiler



3. Distributed Mutual Exclusion (Lamport – Simulation Logic, C++)

Algorithm Used

Lamport's Distributed Mutual Exclusion Algorithm C++

```
#include <iostream>
#include <queue>
using namespace std;
// Structure to represent a request
struct Request {
    int timestamp;
    int processId;
};
int main() {
    queue<Request> requestQueue;
    // Simulating requests from processes
    requestQueue.push({1, 1});
    requestQueue.push({2, 2});
    requestQueue.push({3, 3});
    // Processing requests in order
    while (!requestQueue.empty()) {
        Request r = requestQueue.front();
        requestQueue.pop();
        // Critical Section
        cout << "Process " << r.processId
            << " ENTERS Critical Section at time "
            << r.timestamp << endl;

        cout << "Process " << r.processId
            << " EXITS Critical Section" << endl << endl;
    }
    return 0;
}
```

Output
Command: Run
Platform: Online Compiler

main.cpp

```cpp
    requestQueue.push({3, 3});

    // Processing requests in order
    while (!requestQueue.empty()) {
        Request r = requestQueue.front();
        requestQueue.pop();

        // Critical Section
        cout << "Process " << r.processId
             << " ENTERS Critical Section at time "
             << r.timestamp << endl;

        cout << "Process " << r.processId
             << " EXITS Critical Section" << endl << endl;
    }

    return 0;
}
```

Output

```
Process 1 ENTERS Critical Section at time 1
Process 1 EXITS Critical Section

Process 2 ENTERS Critical Section at time 2
Process 2 EXITS Critical Section

Process 3 ENTERS Critical Section at time 3
Process 3 EXITS Critical Section


=== Code Execution Successful ===
```

Conclusion :-

Peterson's Algorithm successfully enforces exclusive access to the critical section for two concurrent processes. It uses shared variables and turn-based priority to prevent simultaneous entry. The algorithm demonstrates correct handling of race conditions without hardware support. It guarantees progress by allowing one waiting process to enter the critical section.
This experiment confirms its effectiveness for small-scale, two-process synchronization.

Lamport's Bakery Algorithm provides fair access to the critical section using a ticket-based mechanism. Processes are served in the order of their assigned numbers, preventing starvation.
The algorithm scales efficiently to multiple processes in a shared-memory environment. Simulation results show orderly and predictable entry into the critical section.
This validates its suitability for multi-process mutual exclusion scenarios.

Logical timestamps enable consistent ordering of requests in distributed environments. The simulation demonstrates coordination without shared memory or centralized control.
Message-based synchronization ensures correct access to distributed critical sections. Timestamp ordering preserves fairness and prevents conflicting access.
This experiment highlights the practicality of logical clocks in distributed systems.

Sign and Remarks:

R1 (4 Marks)	R2 (4 Marks)	R3 (4 Marks)	R4 (3 Marks)	Total Marks (15 Marks)	Signature